



信息与软件工程学院

综合项目 II 中期报告

课程名称： 异构多核音频处理

课题名称： 基于 DM8168 的语音信号处理

指导教师： 管庆

所在系别： 数字信号处理

执行学期： 2026-2026-2

学生信息：

序号	学号	姓名
1（组长）	2024090912002	陈旺
2	2024090915005	刘福鑫
3	2024090904004	谢昊洋
4	2024090915017	林愉翔
5		
6		

目 录

第一章 综合项目的进展情况	2
1.1 针对工程问题的方案设计	2
1.2 针对工程问题的推理分析	3
1.3 针对工程问题的具体实现	5
1.4 知识技能学习情况	8
第二章 存在问题与解决方案	9
2.1 存在的主要问题	10
2.2 解决方案	12
第三章 前期任务完成度与后续实施计划	14
参考文献	16

第一章 综合项目的进展情况

1.1 针对工程问题的方案设计

本综合设计项目的最终工程目标是：在德州仪器（TI）推出的 DaVinci 架构高端媒体处理器 DM8168（集成 ARM Cortex-A8 核心与 DSP C674x 核心）上，实现语音信号的采集、跨核流转与加速处理。在该系统中，ARM 核心运行 Linux 操作系统，主要负责外围设备控制及上层应用交互；DSP 核心运行 SYS/BIOS 实时操作系统，专职承担高算力需求的算法加速。基于这一复杂的软硬件架构，项目前半学期的核心工程问题被严格界定为：**如何从零构建稳健的跨平台交叉编译体系，并彻底打通、验证 ARM 与 DSP 之间的底层进程间通信（IPC）链路。**针对上述极具挑战性的底层工程问题，本组在中期阶段进行了如下模块化的方案设计：

1. 宿主机-目标板（Host-Target）交叉编译环境架构设计

嵌入式异构系统的宿主机指令集架构（x86）与目标板架构（ARM）存在物理差异，目标板无法直接运行大型编译工具。本组设计了基于虚拟化技术的底层开发架构：在 Windows 物理宿主机上，依托 VMware Workstation 部署 Ubuntu Linux 虚拟机作为主力编译服务器。方案设计利用 VM Tools 与 FUSE 文件系统机制，打通 Windows 与 Linux 之间的底层文件 I/O 屏障，实现大型开发包（EZSDK）与交叉编译工具链（CodeSourcery）的无缝透传与挂载。此设计保证了开发环境的高效性与可移植性。

2. 基于 SysLink 框架的异构双核通信验证方案设计

在正式引入高实时性的音频流之前，必须对底层通信通道的可靠性进行极限验证。本组方案设计选择基于 TI 官方提供的 SysLink IPC 框架进行深度二次开发。为了全面验证数据流与控制流的鲁棒性，我们设计了两套梯度递进的业务逻辑：

- **指令与数值卸载验证（控制流验证）：**设计 ARM 端下发浮点数值，DSP 端

接收指令后在底层调用数学函数执行平方根（开根号）计算并传回结果。此方案旨在验证 SysLink 底层通知机制（Notify）与消息队列（MessageQ）的双向触发是否精准，以及 DSP 核心硬件算术单元的调用情况。

- **连续内存块遍历与加工验证（数据流验证）：**设计 ARM 端从本地文件系统中读取长字符串数据，将其整体写入物理共享内存（SharedRegion）中，随后通知 DSP 端接管该段物理内存指针，通过指针遍历操作将所有小写英文字符统一转换为大写格式。此方案旨在验证双核对大块连续物理内存的读写控制权，以及在 C 语言指针操作下缓冲区的边界封闭性。

3. 底层内核级远程调试体系（Remote Debugging）设计

多多核并发系统中的 Bug（如内存踩踏、指针越界、总线死锁）往往具有极强的隐蔽性，单纯依赖控制台打印的传统方法效率极低。本组方案设计采用基于 GDB（GNU Debugger）与 gdbserver 结合的 C/S 网络调试架构。计划将调试器服务端组件直接部署至 DM8168 目标板的根文件系统中，通过网络与宿主机建立连接，从而为后续排查多核并发环境下的内核级异常提供具有堆栈回溯与内存实时快照功能的“底层视界”。

1.2 针对工程问题的推理分析

上述方案并非凭空设定，而是建立在对异构多核底层硬件机理、Linux 系统资源分配策略以及 C 语言内存模型的深度理论推演与剖析之上。在项目执行前期，本组主要进行了以下三个维度的工程推理：

1. 跨平台编译体系的必要性与寻址映射推理

DM8168 目标板虽具备强大的多媒体处理能力，但其闪存空间与 RAM 资源无法支撑包含庞大链接库的完整 GCC 编译系统运行；且在目标板上直接编译极易由于缺乏必要头文件而导致系统环境崩溃。由此推理得出，采用交叉编译（Cross-

Compilation) 不仅是业界标准, 更是物理环境的必然选择。这就要求我们在 Ubuntu 宿主机上, 必须将系统默认的 gcc 环境彻底隔离, 通过修改环境变量 (export PATH), 将工具链局限并强制指向 arm-none-linux-gnueabi-gcc。同时推理出, 在使用庞大的 TI EZSDK 构建 SysLink 底层驱动包时, 其内部的 Makefile 构建树必然存在极其严密的路径依赖关联, 任何官方预设路径与本地物理挂载路径的不匹配, 都将导致编译链断裂。

2. 异构双核 IPC 通信与共享内存映射的架构推理

ARM 与 DSP 虽然在物理主板上连接着同一片 DDR3 内存芯片, 但它们各自拥有不同的寻址空间, 并分别由主核的 Linux OS 与副核的 SYS/BIOS 实时操作系统进行管理。双核之间若想实现高效的数据互通, 必须建立一套统一的地址转换与信令交互标准。

基于此底层逻辑, 本组推理得出: 双核数据流转必须严格依托 SysLink 框架提供的 SharedRegion 与通信总线模块。SharedRegion 模块通过在系统初始化时划分出专门的共享物理堆内存, 将该段内存的物理首地址同时映射到 ARM 和 DSP 的页表空间中, 使得两端代码能够通过一致的物理偏移量访问相同的数据块。此时, 再配合底层的 IPC 硬件中断触发机制 (Notify), 即可完美打通主副核之间的“信息孤岛”, 确保控制指令与大块字符串能够安全、无损地跨越操作系统壁垒。

3. 连续内存遍历的指针边界与 C 语言状态机推理

在设计“字符串大小写转换”的验证任务时, 需要 DSP 对连续的共享内存进行批量处理。通过对 C 语言内存寻址原理的推演, DSP 在接收到 ARM 传递的首地址指针后, 其内存处理过程本质上是一个基于状态机的循环迭代。对于连续内存的修改, 不仅需要对当前地址中的值进行位运算或算术运算 (如 ASCII 码偏移减去 32 以转换为大写), 更核心的是必须在每一次迭代后, 手动驱动指针在物理

空间中发生确切步长的位移（Offset）。任何丢失步长控制（如缺失 `ptr++` 操作）的循环逻辑，都会导致处理器永远停留在初始物理地址上，陷入对同一内存单元无限次覆写的死循环状态。这一严密的推演，也在后续的实机开发中精准地预言并解释了我们在代码中踩到的“转换异常 Bug”。

1.3 针对工程问题的具体实现

本组在中期阶段严格对照课程大纲与项目进阶计划，分步推行底层公路的铺设与框架通信的重构，完整实现了从宿主机虚拟化到目标板异构算法级联调的全部既定任务（阶段一至阶段六）。具体工程落地细节实现如下：

1.3.1 虚拟化编译宿主机搭建与底层共享文件系统挂载（阶段一）

在跨平台开发体系的初始构建期，本组在物理宿主机上部署了 `VMware Workstation` 虚拟化平台，并配置了 `Ubuntu Linux` 操作系统作为异构多核程序的编译宿主服务器。在搭建终端控制台与辅助传输软件（如 `SecureCRT` 等）时，系统底层报出缺乏 `.Net Framework 2.0/3.0` 内核驱动组件的严重依赖异常，且直接下载离线包导致安装程序死锁。本组通过走查操作系统底层机制，转入 `Windows` 控制面板的“启用或关闭 `Windows` 功能”底座，勾选对应服务交由微软底层组件自动托管下载与配置，彻底扫清了宿主机端的工具链障碍。

为了将体积庞大的核心开发包 `ezsdk.tar.gz` 与高级交叉编译器 `CodeSourcery.tar.gz` 导入 `Linux` 虚拟机，本组启用了 `VMware` 的共享文件夹服务。但在实机挂载过程中，虚拟机内核未能正确自动加载文件系统，导致 `/mnt/hgfs/` 目标挂载目录下未能浮现出物理机映射的文件。针对此共享文件系统失效故障，本组深入探究了 `Linux` 空间下的 `FUSE`（用户空间文件系统）管理机制，在终端交互中手动调用了底层文件挂载高级指令：

```
sudo vmhgfs-fuse .host:/share /mnt/hgfs
```

在克服了初期的权限拒绝与挂载树断裂报错后，通过重新初始化虚拟化管理

平台的底座属性，成功将两大核心压缩包透传至 Linux 编译系统内部，并通过 `tar -zxvf` 级联指令将其完整解压部署至本机的本地工作目录 `/home/cw/` 空间内。

1.3.2 交叉编译工具链配置与基础单核程序验证（阶段二、三）

为使工作主机能够编译出 DM8168 目标板可以直接读取的 ARM 架构执行代码，必须进行交叉编译环境变量的注入。本组通过重构环境配置文件，将解压后的 CodeSourcery 工具链中的 `bin` 物理目录，通过 `export PATH` 语句追加至全局系统路径树中。

为了对该交叉编译环境的连通性与链接正确性进行闭环验证，本组编写了经典的单核基准测试程序 `helloworld.c`。在编译阶段，彻底隔离宿主机本地的 `gcc`，调用交叉编译器进行链接：

```
arm-none-linux-gnueabi-gcc helloworld.c -o helloworld
```

成功生成了 ARM 架构下的 ELF 二进制可执行文件。随后，通过挂载的网络文件系统（NFS）将该程序导入 DM8168 开发板的目标 Linux 文件系统中进行实机加载。程序在开发板终端完美输出预期字符，标志着项目正式通过了对目标板、内核、U-Boot 以及基本单核运行环境的基础认知与验证阶段。

1.3.3 SysLink 构建树重构与多核基准框架编译（阶段四）

在进入双核通信框架的编译阶段时，由于 TI 官方提供的 SDK 包年代久远，其内部各个模块的依赖关系错综复杂，直接执行 `make` 引发了全量路径死锁报错。本组深入源码控制层 `/syslink_2_20_00_14` 路径，对顶层及各子模块的全局编译配置文件 `products.mak` 展开了精准的外科手术式重构：

1. **绝对路径变量重定向：** 逐行查阅宏定义，将第 49 行、103 行、109 行、116 行等关键变量（包括 ARM 工具链路径、DSP BIOS 路径、IPC 组件路径等）硬编码的官方作者私人路径（如 `/home/user/...`），批量变更为符合当前编译服务器物理真实的绝对路径 `/home/cw/...`。
2. **语法冗余剔除：** 在追踪深层 Makefile 解析错误时，本组敏锐捕捉到 `/ex04/products.mak` 配置文件第 60 行处，其预设路径的末尾存在一个极其

隐蔽且冗余的斜杠 (/) 语法错误，导致构建树在拼接字符串时路径失效，将其果断删除后扫清了路径解析障碍。

3. **权限链闭环修复：** 在保存上述重大配置修改时，系统抛出由于文件属主为 Root 锁定而导致的只读拒绝异常。本组严格遵循 Linux 安全机制，拒绝进行高风险的暴破强写，选择删除原受损文件夹，以当前标准用户身份重新完整解压 SDK 压缩包，使整个工程树完美继承了正确的读写权限链。

通过重构该编译树，本组在宿主机终端成功执行了全量编译，生成了 ARM 端主控程序 (app_host) 与 DSP 端执行体镜像 (server_dsp.xe674)。将其部署至目标板的物理 debug 文件夹后，执行官方引导脚本 ./run.sh，成功通过内核加载了 syslink.ko 驱动，并利用 slaveloader 顺利唤醒了副核 DSP，跑通了多核基准框架。

1.3.4 异构通信例程算法级重构与地址迭代 Bug 修复（阶段五）

为了吃透 SysLink 框架下的双核协同机制，本组在打通基准框架的前提下，对官方通信例程的数据链路层与业务层代码进行了深度的二次重构开发，实现了两个层面的异构算法验证：

- **业务一：数学开根号运算的异构卸载验证**

修改了 DSP 侧的处理器 Task 代码，在其内部接驳了数学运算库。设计由 ARM 端通过消息队列下发包含浮点型输入参数的数据包，DSP 端捕获中断后唤醒本地高优先级的实时任务，调用硬件浮点算术单元执行平方根高精度计算，并将计算结果通过 SharedRegion 物理共享内存完美回传至 ARM 端进行控制台打印。此项任务的成功，切实完成了控制指令级跨核卸载的鲁棒性验证。

- **业务二：大块连续共享内存读取与字符大小写转换验证**

该业务旨在验证双核对大容量连续物理内存的批量读写权限。设计由 ARM 端从本地 Linux 文件系统中读取一段长字符串，将其完整写入 SharedRegion 共享堆内存中，随后触发 IPC Notify 硬件中断通知副核。DSP 端接管该物理共享指针后，遍历内存空间，将其中所有小写英文字符转换为大写格式。

- **核心指针偏移 Bug 的走查与修复：**

在首轮联合调试测试中，系统暴露出严重的业务运行异常：ARM 端下发长字符串并引发 DSP 处理后，回传打印的结果显示**仅有整个字符串的第一个字符被成功转换为了大写，其余后续字符完全保持原样**。本组针对该异常展开了严密的代码断点走查与逻辑推演，最终将 Bug 定位在 DSP 端的 C 语言处理算法内部。经查，DSP 核心在执行完当前内存单元的 ASCII 码值偏移（`*ptr = *ptr - 32`）后，代码逻辑中遗漏了驱动内存地址游标向后推移的关键物理指令（即缺失了 `ptr++` 操作）。这导致 DSP 的 while 循环在整个执行周期内，都在原地反复覆写共享内存的首字节。本组在循环体内补齐了精准的指针步长迭代逻辑后，系统成功实现了整块连续内存大数据的批量、无损跨核转换，数据流转通路彻底宣告打通。

1.3.5 目标板级 GDB 远程网络调试体系部署（阶段六）

为了给后续高负载、多业务的复杂程序开发搭建强有力的防错排雷底座，本组在中期节点前提前布局并竣工了系统级的远程联调环境。本组深入交叉编译工具链的底层库目录中，精准提取出了专用于目标板嵌入式架构的 `gdbserver` 核心执行组件。

通过目标板根文件系统的映射通道，将 `gdbserver` 手动拷贝部署至开发板根文件系统的 `/usr/bin/` 核心系统路径下。随后在开发板的 Linux Shell 环境中配置了持久化的 `export PATH` 环境变量，使得实机能够随时唤醒调试监听端口。该体系的成功搭建，打通了宿主机高级 GDB 源码级调试器与 DM8168 目标板内核之间的 Socket 网络联调通路，使本组具备了在后续开发中实时截获多核物理堆栈、监视物理共享内存快照的强大工程能力。

1.4 知识技能学习情况

通过项目前半程高强度的工程磨练，本组成员在理论视野、技术深度以及实战工程素养上均获得了蜕变式的成长，具体体现在以下三个方面：

1. 工业级构建系统与 Linux 权限体系的深度掌控

成员们彻底摆脱了传统集成开发环境（IDE）“一键编译”的思维依赖，深入理解了嵌入式 Linux 系统下的文件属主、Root 锁定、用户组读写隔离等内核安全机制。通过对多份 `products.mak` 文件的重构与语法纠错，熟练掌握了大型复杂工程构建树中的宏替换规则、隐式路径拼接以及复杂的依赖链传递逻辑，具备了独立排查和重塑底层 Makefile 系统的工程能力。

2. 异构多核通信机理与页表共享内存的初步领悟

跨越了传统单核 MCU 编程的固有局限，深刻领会了异构多核分布式系统的设计美学。通过实机操控 SysLink 框架，不仅在理论上弄懂了 MessageQ 消息控制流与 SharedRegion 物理数据流相互解耦的先进架构，更在实践中掌握了如何在相互物理隔离、运行完全不同操作系统的双核之间，建立可靠的物理页表映射与中断同步握手协议。

3. 系统级 C 语言内存指针的严谨管控与代码走查素养

在排查并修复“DSP 大写转换仅处理首字母”的工程 Bug 过程中，全组成员切身体会到了系统级 C 语言开发中“指针即物理地址，步长即生死线”的深刻内涵。深刻认识到在多核共享内存的批量操作中，任何一个微小的游标位移缺失都将导致物理内存的局部瘫痪甚至引发内核 Panic。这一过程极大地锻炼了成员们严密的逻辑推理能力与面向底层的严谨编程素养。

第二章 存在问题与解决方案

在项目前半程的环境搭建、SysLink 编译树重构以及多核基础通信例程验证的“排雷”工程中，由于软硬件体系年代久远、系统间隔离严格、跨平台寻址极其敏感，本组遭遇了涉及宿主机虚拟化、编译构建树、Linux 文件系统权限、以及

底层 C 语言指针寻址等层面的诸多典型工程障碍。以下为 中期前发生的六大核心问题及其技术破局过程的详实记录：

2.1 存在的主要问题

2.1.1 终端环境驱动依赖组件（.Net Framework）网络下载死锁问题

在配置开发板控制台终端辅助软件（如 SecureCRT 及相关串口驱动）的初期阶段，宿主机系统底座抛出依赖冲突，提示必须安装 .Net Framework 2.0/3.0 运行时组件。本组直接从网络下载了独立的离线安装包进行部署，但由于物理宿主机内部的网络安全策略隔离与微软服务器握手超时，导致安装进程长时间卡死在进度条零点，引发了严重的终端软件环境搭建死锁。

2.1.2 宿主机与虚拟机物理隔离导致的文件共享系统（HGFS）挂载失效

为了将物理机上下载的数 GB 体积的开发核心包（ezsdk.tar.gz）和交叉编译器（CodeSourcery.tar.gz）导入 Linux 编译机，本组在 VMware 中开启了共享文件夹（Shared Folders）服务，映射路径为物理机上的 share 目录。然而，在启动虚拟机进入 Ubuntu 系统后，Linux 内核未能自动加载对应模块，导致虚拟机内核层 /mnt/hgfs/ 挂载点目录下呈现完全空白状态，宿主机与虚拟机之间的大文件透传链路彻底断裂。

2.1.3 SysLink 全局编译树（products.mak）硬编码路径导致的工具链寻址断裂

在进入 SysLink 跨核 IPC 基础框架的首次编译阶段，本组在终端执行 make 全量构建指令，系统瞬间爆出数百行致命错误，提示无法定位编译器、BIOS 系统内核及相关头文件。经跟踪构建脚本，发现德州仪器（TI）官方 EZSDK 开发包内的核心配置文件 products.mak 中，大量存在原官方作者本机的私人绝对路径硬编码（如 /home/user/...），特别是在第 49 行、103 行、109

行、116 行等控制变量关键位置，导致交叉编译构建树由于路径污染而全面瘫痪。

2.1.4 示例 ex04 子模块编译路径中隐蔽的冗余斜杠 (/) 语法错误

在成功修复了全局路径变量后，本组对 SysLink 自带的子模块例程进行级联编译。当构建树推演至 /ex04/ 目录时，编译再次发生中断，抛出无法识别目标路径的异常。经过对 /ex04/products.mak 文件的语法走查，发现其第 60 行处，关于某个关键依赖组件的路径字符串末尾，硬编码了一个极其隐蔽的冗余斜杠 (/)。这一微小的语法疏漏导致 Makefile 在进行后续字符串拼接时，拼出了类似 // 的非法寻址路径，阻断了整个工程树的编译自顶向下传递。

2.1.5 Linux 文件系统权限继承 (Root 锁定) 导致的文件只读保护冲突

在试图对上述全局及子模块的 products.mak 配置文件进行文本修正与保存时，Ubuntu 系统强行拦截并抛出“Permission Denied (拒绝访问)”的只读保护异常。经排查文件属性，发现由于初期解压 SDK 开发包时使用了不当的提权操作，导致整个核心开发工程文件夹的绝对所有权被系统超级用户 (Root) 彻底锁定。普通开发用户在标准 shell 下无权对其进行任何修改、覆盖或写入，使得配置更新工作陷入僵局。

2.1.6 DSP 端字符大写转换算法中因“指针空间位移缺失”导致的数据覆写异常

在成功打通多核通信链路后，本组重构了双核例程以验证大块共享内存的数据流转。业务逻辑要求 ARM 端将长字符串写入物理共享内存 (SharedRegion) 并触发中断，DSP 端接管指针后遍历内存将小写字母转换为大写。然而，在实机联合调试的首轮测试中，系统报出严重的运行偏差：ARM 终端回传打印的结果显示，仅有整个字符串的第一个字符被成功转换为了大写，而后续的大段文本字符完全保持原样，没有发生任何改变。

2.2 解决方案

2.2.1 引入 Windows 功能底座的静默托管下载策略

针对 .Net Framework 离线安装死锁问题，本组放弃了高风险的第三方独立安装包，转而深入 Windows 操作系统的控制底座。通过进入“控制面板”->“程序和功能”->“启用或关闭 Windows 功能”核心面板，在系统组件树中精准勾选了对应的 .Net Framework 3.0（包括 2.0）选框，将下载与安全校验工作彻底移交给 Windows 系统底层的原生补丁分发服务。由系统执行静默托管下载与内核级挂载，成功排除死锁，确保了串口控制台软件的顺利驱动。

2.2.2 基于 FUSE 文件系统的强制内核空间映射重构

针对虚拟机 /mnt/hgfs/ 共享文件夹空白的问题，本组深入探究了 Linux 空间下的 FUSE（用户空间文件系统）底层驱动机制，在 Ubuntu 终端中手动键入了强力内核挂载指令：

```
sudo vmhgfs-fuse .host:/share /mnt/hgfs
```

在执行初期，系统由于挂载树断裂上报了权限拒绝异常。本组采取了先在 VMware 管理端彻底关闭共享、清理内核失效挂载点，随后重新初始化虚拟化平台的映射属性并赋予全局安全信任。再次执行上述 FUSE 指令后，成功在 /mnt/hgfs/ 目录下浮现出 Windows 端的 share 文件夹，扫清了大体积软件包的搬运阻碍。

2.2.3 针对构建树多处硬编码地雷的绝对路径变量更正

为了修复 SysLink 的多核交叉编译环境，本组利用高级文本编辑器对根目录下的 products.mak 配置文件展开了精准的外科手术式路径重组。对照本地交叉编译工具链的实际部署节点，将该文件第 49 行、103 行、109 行、116 行等控制核心路径的变量，逐一将过期的 /home/user/... 垃圾字段强行变更为本机当前的绝对真实路径 /home/cw/...（涵盖了 XDC_INSTALL_DIR、

BIOS_INSTALL_DIR、IPC_INSTALL_DIR 等)，彻底将断裂的编译寻址链条重新咬合。

2.2.4 级联子模块路径字符串的微观语法纠错

针对 /ex04/products.mak 第 60 行触发的拼接寻址错误，本组摒弃了宏观的环境排查，聚焦于微观的编译脚本走查。精准定位到那处多余的斜杠 (/) 后，使用文本编辑器将其果断予以删除。这一行微观语法的纠正，使得 Makefile 在执行下游路径级联拼接 (PATH_A+PATH_B) 时，恢复了正确的格式标准，从而实现了 make 指令在子模块工程内的顺畅放行。

2.2.5 遵循安全规范的用户空间读写权限链闭环重塑

面对不可逾越的 Root 权限文件锁，本组没有采取高风险的 `sudo chmod 777` 暴力提权（这会导致后续编译生成的二进制文件带有不可控的用户属主属性），而是遵循 Linux 系统的标准安全工程规范。本组调用 `sudo rm -rf` 彻底删除了原受损且属主错乱的开发包目录，切换回标准开发用户 (cw) 身份，重新调用解压指令将 `ezsdk.tar.gz` 纯净地释放到本地 /home/cw/ 目录下。此举使得新生成的所有子文件和编译脚本完美、合规地继承了当前标准用户的读写权，彻底根除了只读保护冲突。

2.2.6 DSP 端处理算法循环体内的指针步长自增修复

该问题的技术本质是：DSP 处理器在操作连续物理共享内存时，缺失了空间维度上的物理游标推进机制。本组通过对 DSP 侧 C 语言处理 Task 展开严格的代码逻辑走查，发现算法在通过 `*ptr=*ptr-32`；完成当前内存单元的大写转换后，循环体内竟然遗漏了将指针地址向后推移的关键物理指令。这导致 CPU 在整个 while 遍历周期内，都在原地踏步地对共享内存的首字节进行无限次盲目覆写。本组通过在循环体末尾补齐了丢失的指针游标自增控制代码：

```
ptr++; // 强制驱动物理地址游标向后推移一个字节步长
```

该项修复赋予了副核 DSP 处理器按字节步长顺利向后开辟空间寻址的物理能力。再次进行实机联合测试，系统成功实现了整块物理共享内存内长字符串数据的全量、无损跨核转换验证，标志着双核间的异构通信基座至此全线竣工。

第三章 前期任务完成度与后续实施计划

3.1 前期任务完成度评估

截至本中期检查节点，本组已 **100%完成**了课程教学大纲与进度安排表中规划的“第一部分：基础知识以及开发环境”与“U-boot、Kernel 编译与 Syslink 例程架构学习”的全部核心建设指标。

各项子任务的达成度量化如下表所示：

阶段划分	核心大纲任务要求	本组实际完成情况	达成度
阶段一	虚拟化环境配置与控制台串口驱动打通	成功部署 VMware Ubuntu 系统，排除 .Net 驱动障碍，挂载 HGFS 成功	100%
阶段二/三	交叉编译链配置与单核 Linux 应用验证	注入 CodeSourcery 环境变量，通过 HelloWorld 程序跑通目标板	100%
阶段四	驱动编译与 SysLink 基础多核框架运行	彻底重构多份 products.mak 地雷，通过 run.sh 成功唤醒 DSP	100%
阶段五	SysLink 例程业务级修改与异构验证	独立完成 DSP 端开根号卸载与大块内存大写转换指针 Bug 修复	100%
阶段六	系统级高级联调工具链部署	成功提取并移植 gdbserver 至开发板，打通 Socket 网络调试链路	100%

3.2 后续实施计划（期中至期末）

既然底层的交通公路（SysLink 异构物理通信架构）已经全线修通，在下半学期的深水区开发中，项目将正式按照大纲规划，进入大流量、高实时性的“真实语音流上路”阶段。具体的时间表与技术攻关路线规划如下：

1. 第 9 周 - 第 12 周：音频外设驱动集成与数据管线（Data Pipeline）打通

- **核心目标：** 在 ARM 端的 Linux 操作系统中正式接驳 ALSA（Advanced Linux Sound Architecture）高级音频架构驱动。
- **攻关要点：** 编写标准的音频采集（Capture）与播放（Playback）控制模块，设定工业级的采样率（48000Hz）、量化位深（S16_LE）与双声道参数。攻克如何将麦克风采集到的连续 PCM 音频裸流，以定长数据块（Block）的形式安全地推入到我们前期验证成功的 SharedRegion 共享内存堆中。

2. 第 13 周 - 第 14 周：多线程并发模型引入与双核高可靠流控流架构设计

- **核心目标：** 解决分时操作系统调度抖动对高实时音频流造成的断流与撕裂问题。
- **攻关要点：** 在 ARM 端的应用层代码中引入 POSIX 多线程（pthread）并发架构，派生出完全解耦的独立录音线程与独立播放线程，切断由于 I/O 阻塞造成的时序强耦合。同时，利用跨核消息队列与本地信号量（Semaphore）构建闭环流控，并在代码中引入预充水机制（Start Threshold）与硬件级异常重置机制，确保系统在高负载下始终保持双声道对齐。

3. 第 15 周 - 第 16 周：上位机图形界面（QT）集成与异构全链路系统交付

- **核心目标：** 遵照课程第三阶段规划，构建可视化的人机交互控制面板。
- **攻关要点：** 在 ARM 核心的应用层引入 QT 图形用户界面开发框架。编写 GUI 界面，通过信号与槽机制封装底层的 ALSA 状态机与 SysLink 通信信令。在图形化面板上实时展现音频的波形时域图、支持动态下发加速处理指令，最终完成 72 小时连续抗压拷机测试，交付完整的工业级异构多核语

音信号处理展示系统。

参考文献

- [1] 电子科技大学. 《2026 春季学期-综合设计 II-异构多核（基于 DM8168 的语音信号处理）-计划安排 v1.4》. 2026.
- [2] Texas Instruments. *SysLink User's Guide and IPC Framework Reference (v2.20)*. 2012.
- [3] Texas Instruments. *TMS320DM8168 DaVinci Digital Media Processor Technical Reference Manual*. 2011.
- [4] Free Software Foundation. *Debugging with GDB: The GNU Source-Level Debugger*. 2020.